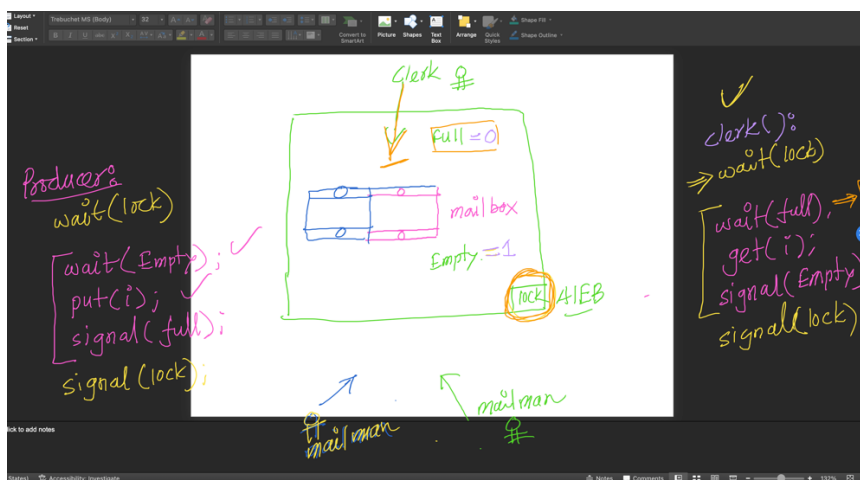
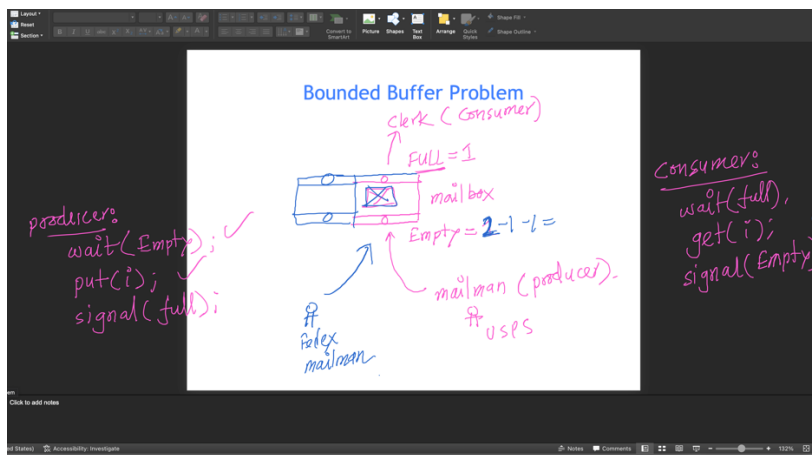
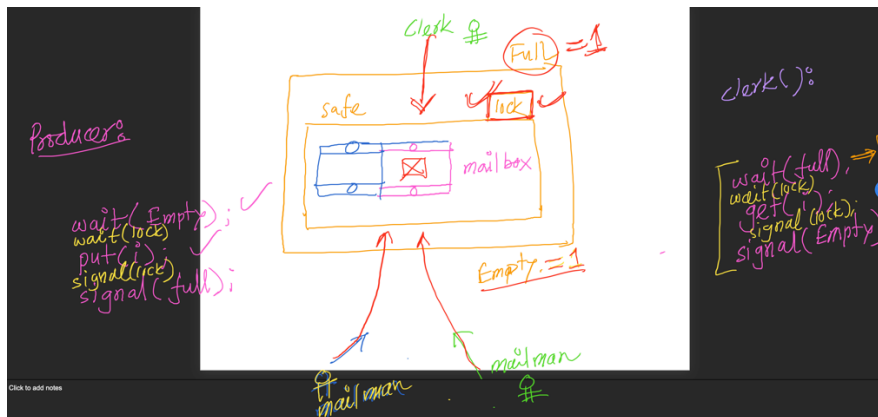


Semaphore

Bounded Buffer Problem:

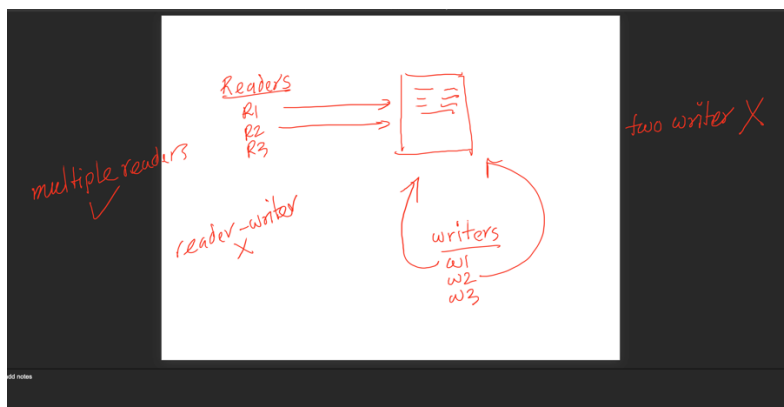
- Shared buffer with N slots to store at most N items
- Producer processes data items and puts into the buffer
- Consumer gets the data items from the buffer
- Variable empty keeps number of empty slots in the buffer
- Variable full keeps number of full items in the buffer





Readers Writers Problem:

- Multiple Readers and writers concurrently accessing the same database.
- Multiple Readers accessing at the same time --> OK
- When there is a Writer accessing, there should be no other process accessing at the same time.



- The structure of a **writer process**

```
do {
    wait (wrt) ;

    //  writing is performed

    signal (wrt) ;
} while (true)
```

- The structure of a **reader process**

```
do {
    wait (mutex) ;
    readercount ++ ;
    if (readercount == 1) wait (wrt) ;
    signal (mutex) ;

    // reading is performed

    wait (mutex) ;
    readercount -- ;
    if readercount == 0) signal (wrt) ;
    signal (mutex) ;
} while (true)
```

What is dining philosophers problem?

- Five philosophers spend their time eating and thinking.
 - They are sitting in front of a round table with spaghetti served.
 - There are five plates at the table and five chopsticks set between the plates.
 - Eating the spaghetti requires the use of two chopsticks which the philosophers pick up one at a time.
 - Philosophers do not talk to each other.
- Semaphore chopstick [5] initialized to 1

What happens to dining philosophers when each runs following code.

- The structure of Philosopher i :
 Do {
 wait (chopstick[i]);
 wait (chopStick[(i + 1) % 5]);

 // eat
 signal (chopstick[i]);
 signal (chopstick[(i + 1) % 5]);

 // think
 } while (true) ;

It may result in deadlock.

How to prevent deadlock in dining philosopher problem?

- Allow philosopher to pick up his chopsticks only if both chopsticks are available (i.e. in critical section)
- Use an asymmetric solution: an odd philosopher picks up first his left chopstick and then his right chopstick; and vice versa

DEADLOCKS

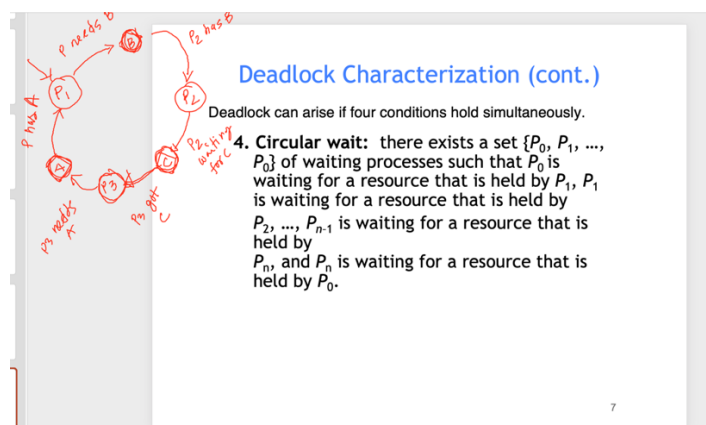
What is a deadlock?

- A set of blocked processes each holding a resource and waiting to acquire a resource held by another process in the set.
- Example
 - System has 2 disk drives.
 - P_1 and P_2 each hold one disk drive and each needs another one.
- Example
 - semaphores A and B , initialized to 1

P_0	P_1
$wait(A);$	$wait(B)$
$wait(B);$	$wait(A)$

Deadlock characteristics

1. **Mutual exclusion:** nonshared resources; only one process at a time can use a specific resource
2. **Hold and wait:** a process holding at least one resource is waiting to acquire additional resources held by other processes
3. **No preemption:** a resource can be released only voluntarily by the process holding it, after that process has completed its task

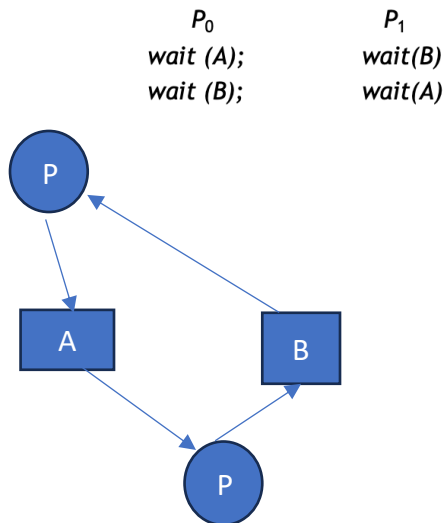


Notations for Resource Allocation graph

- Process 
- Resource Type with 4 instances 
- P_i requests instance of R_j 
- P_i is holding an instance of R_j 

Do we have a deadlock in the following example?

- semaphores A and B, initialized to 1



At time $t = 0$, p_0 runs *wait(A)* and p_1 runs *wait(B)*.

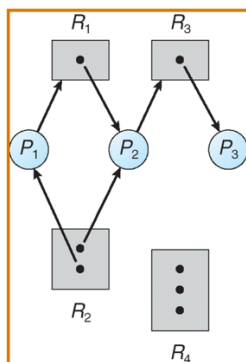
Initially, both A and B free. So, P_0 gets A and p_1 gets B.

At time $t = 1$, p_0 runs *wait(B)* and p_1 runs *wait(A)*.

But both of the resources are already acquired.

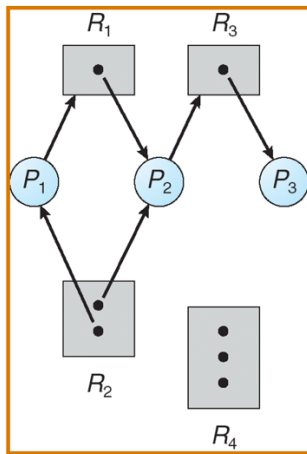
So, p_0 and p_1 cannot move forward. So, deadlock.

Example of resource allocation graph



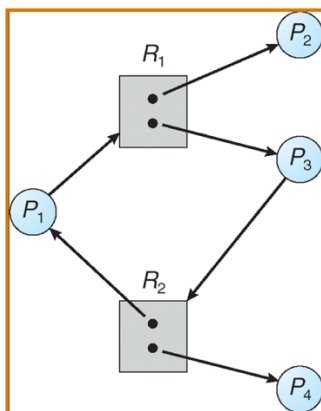
- If graph contains no cycles $\Rightarrow$ no deadlock.
- If graph contains a cycle $\Rightarrow$ there may be a deadlock
 - if only one instance per resource type, then deadlock.
 - if several instances per resource type, possibility of deadlock.

Do we have a deadlock in the following graph?



→ No Cycle, no Deadlock

Do we have a deadlock in the following graph?



When several instances of resource are available, only cycle cannot guarantee deadlock.

Here p2 has r1 and runs. When p2 is done, it releases r1.

P1 has r2 and waiting for r1. Now r1 is available (p2 released it). P1 can get it and run.

When p1 is done it releases both r2 and r1.

P3 has r1, and now can get r2 and run.

P4 can run.

So no deadlock

Deadlock detection:

If all the resources have single instance,
then look for cycle in the resource allocation graph.
If cycle exit,
Then deadlock might happen.

Cycle detection takes n^2 operations.

If several instances of resources,
Use banker's algorithm.

Deadlock detection algorithm:

1. Let *Work* and *Finish* be vectors of length *m* and *n*, respectively Initialize:
 - (a) *Work* = *Available*
 - (b) For $i = 0, 1, 2, \dots, n-1$, *Finish*[*i*] = false.
2. Find an index *i* such that both:
 - (a) *Finish*[*i*] == false
 - (b) $Request_i \leq Work$
 If no such *i* exists, go to step 4.
3. *Work* = *Work* + *Allocation*_{*i*}
Finish[*i*] = true
 go to step 2.
4. If *Finish*[*i*] == false, for some $i, 0 \leq i \leq n-1$, then the system is in deadlock state.
 Moreover, if *Finish*[*i*] == false, then *P_i* is deadlocked.

Algorithm requires an order of $O(m \times n^2)$ operations to detect whether the system is in deadlocked state.

Practice examples of banker's algorithm

Example of banker's algorithm

- Five processes *P₀* through *P₄*; three resource types *A* (7 instances), *B* (2 instances), and *C* (6 instances).
- Snapshot at time *T₀*:

	<u>Allocation</u>			<u>Request</u>			<u>Available Work</u>		
	<i>A</i>	<i>B</i>	<i>C</i>	<i>A</i>	<i>B</i>	<i>C</i>	<i>A</i>	<i>B</i>	<i>C</i>
<i>P₀</i>	0	1	0	0	0	0	0	0	0
<i>P₁</i>	2	0	0	2	0	2			
<i>P₂</i>	3	0	3	0	0	0			
<i>P₃</i>	2	1	1	1	0	0			
<i>P₄</i>	0	0	2	0	0	2			

Sequence $\langle P_0, P_2, P_3, P_1, P_4 \rangle$ will result in *Finish*[*i*] = true for all *i*.

$$\text{Finish} = \begin{array}{|c|c|c|c|c|} \hline 0 & 0 & 0 & 0 & 0 \\ \hline \end{array} \\ \begin{array}{c} P_0 \ P_1 \ P_2 \ P_3 \ P_4 \end{array}$$

for P_0 req $\begin{bmatrix} 0 & 0 & 0 \end{bmatrix} \leq \begin{bmatrix} 0 & 0 & 0 \end{bmatrix}$ work.

So, P_0 runs. When it is done $\rightarrow$

$$\text{work} = \begin{array}{|c|c|c|} \hline 0 & 0 & 0 \\ \hline \text{work} \end{array} + \begin{array}{|c|c|c|} \hline 0 & 1 & 0 \\ \hline \text{Allocation} \end{array} = \begin{array}{|c|c|c|} \hline 0 & 1 & 0 \\ \hline \text{work} \end{array}, \quad \text{finish} = \begin{array}{|c|c|c|c|c|} \hline 1 & 0 & 0 & 0 & 0 \\ \hline \end{array} \\ \begin{array}{c} P_0 \ P_1 \ P_2 \ P_3 \ P_4 \end{array}$$

Again, P_2 req $\begin{bmatrix} 0 & 0 & 0 \end{bmatrix} \leq \begin{bmatrix} 0 & 1 & 0 \end{bmatrix}$ work.

So, P_2 runs, when it is done, $\text{work} = \begin{array}{|c|c|c|} \hline 0 & 1 & 0 \\ \hline \text{work} \end{array} + \begin{array}{|c|c|c|} \hline 3 & 0 & 3 \\ \hline \text{Allocation} \end{array} = \begin{array}{|c|c|c|} \hline 3 & 1 & 3 \\ \hline \text{work} \end{array}$

Again, P_3 req $\begin{bmatrix} 1 & 0 & 0 \end{bmatrix} \leq \begin{bmatrix} 3 & 1 & 3 \end{bmatrix}$ work,

So, P_3 can run, when it is done, $\text{work} = \begin{array}{|c|c|c|} \hline 3 & 1 & 3 \\ \hline \text{work} \end{array} + \begin{array}{|c|c|c|} \hline 2 & 1 & 1 \\ \hline \text{Allocation} \end{array} = \begin{array}{|c|c|c|} \hline 5 & 2 & 4 \\ \hline \text{work} \end{array}$

Again P_1 req $\begin{bmatrix} 2 & 0 & 2 \end{bmatrix} \leq \begin{bmatrix} 5 & 2 & 4 \end{bmatrix}$ work,

So, P_1 can run. when it is done, $\text{work} = \begin{array}{|c|c|c|} \hline 5 & 2 & 4 \\ \hline \end{array} + \begin{array}{|c|c|c|} \hline 2 & 0 & 0 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 7 & 2 & 4 \\ \hline \end{array}$

Again, P_4 req $\begin{bmatrix} 0 & 0 & 2 \end{bmatrix} \leq \begin{bmatrix} 7 & 2 & 4 \end{bmatrix}$ work.

So, P_4 can run. when it is done, $\text{work} = \begin{array}{|c|c|c|} \hline 7 & 2 & 4 \\ \hline \end{array} + \begin{array}{|c|c|c|} \hline 0 & 0 & 2 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 7 & 2 & 6 \\ \hline \end{array}$

So, Everyone can run. There is no deadlock!

$$\text{finish} = \begin{array}{|c|c|c|c|c|} \hline 1 & 1 & 1 & 1 & 1 \\ \hline \end{array} \\ \begin{array}{c} P_0 \ P_1 \ P_2 \ P_3 \ P_4 \end{array}$$

Resource request algorithm for process p_i :

Let $Request_i$ be the request vector for process P_i .

If $Request_i[j] = k$ then process P_i wants k instances of resource type R_j .

1. If $Request_i \leq Need_i$ go to step 2. Otherwise, raise error condition, since process has exceeded its maximum claim.
2. If $Request_i \leq Available$, go to step 3. Otherwise P_i must wait, since resources are not available.
3. Pretend to allocate requested resources to P_i by modifying the state as follows:
 $Available = Available - Request_i;$
 $Allocation_i = Allocation_i + Request_i;$
 $Need_i = Need_i - Request_i;$
 - If safe $\Rightarrow$ the resources are allocated to P_i .
 - If unsafe $\Rightarrow P_i$ must wait, and the old resource-allocation state is restored

How consider the previous example.

Example of banker's algorithm

- Five processes P_0 through P_4 ; three resource types A (7 instances), B (2 instances), and C (6 instances).
- Snapshot at time T_0 :

	<u>Allocation</u>			<u>Need</u>			<u>Available</u>		
	A	B	C	A	B	C	A	B	C
P_0	0	1	0	7	4	3	2	3	0
P_1	3	0	2	0	2	0			
P_2	3	0	1	6	0	0			
P_3	2	1	1	0	1	1			
P_4	0	0	2	4	3	1			

Sequence $\langle P_0, P_2, P_3, P_1, P_4 \rangle$ will result in $Finish[i] = \text{true}$ for all i .

Executing safety algorithm shows that sequence $\langle P_1, P_3, P_4, P_0, P_2 \rangle$ satisfies safety requirement.

1. Can request for (3,3,0) by P_4 be granted?
2. Can request for (0,2,0) by P_0 be granted?

1. Request 3,3,0 is greater than available 2,3,0. So p_4 needs to wait.

2. $P_0 \text{ req } [0, 2, 0] \leq \text{available } [2, 3, 0]$
 Pretend to allocate resources to P_0 .

$$\begin{array}{r} \text{available} = [2, 3, 0] \\ \text{req} = [0, 2, 0] \\ \hline [2, 1, 0] \end{array}$$

$$\text{work} = \text{available} = [2, 1, 0]$$

No run safety algorithm:

there is no process that honors
 $\text{need} < \text{work}$

so it is not safe. P_0 waits.

$$\begin{aligned} \text{available} &= \text{available} - \text{req} \\ &= [2, 3, 0] - [0, 2, 0] \\ &= [2, 1, 0] \\ \text{allocation} &= \text{allocation} + \text{req} \\ &= [0, 1, 0] + [0, 2, 0] \\ &= [0, 3, 0] \\ \text{need} &= \text{need} - \text{req} \\ &= [7, 4, 3] - [0, 2, 0] \\ &= [7, 2, 3] \end{aligned}$$

Virtual Memory

Page replacement algorithms

First-In-First-Out (FIFO) Algorithm

- Reference string: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5
- 3 frames (3 pages can be in memory at a time per process)

1	4	5
2	1	3
3	2	4

9 page faults

First-In-First-Out (FIFO) Algorithm

- Reference string: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5
- 3 frames (3 pages can be in memory at a time per process)

1	1	4	5
2	2	1	3
3	3	2	4

9 page faults

- 4 frames

1	1	5	4
2	2	1	5
3	3	2	
4	4	3	

10 page faults

- FIFO Replacement - **Belady's Anomaly**
 - more frames $\Rightarrow$ more page faults

Optimal Algorithm

- Replace page that **will not be used for longest period of time**
- 4 frames example

1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

1	4
2	
3	
4	5

6 page faults

Least Recently Used (LRU)

- Reference string: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5
- 4 frames example:

1	5
2	
3	5 4
4	3

8 page faults

Exercise

- Consider the following page-reference string:

1, 2, 3, 4, 4, 3, 2, 1, 5, 6, 2, 1, 2, 3, 7, 8, 3, 2, 1, 5

Assuming 4 memory frames and LRU, LFU, or Optimal page replacement algorithms, how many page faults, page hits, and page replacements would occur? Show your page assignments to frames.

1	2	3	4	4	3	2	1	5	6	2	1	2	3	7	8	3	2	1	5
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2
3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3
4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4

of page faults: 11

of page hits: 9

of page replacements: 7

Optimal: Replace page that **will not be used for longest period of time**

Example

- Consider the following page-reference string:

1, 2, 3, 4, 4, 3, 2, 1, 5, 6, 2, 1, 2, 3, 7, 8, 3, 2, 1, 5

Assuming 4 memory frames and LRU-Clock algorithm, how many page faults, page hits, and page replacements would occur? Show your page assignments to frames. .

Consider the following rules:

1. When a page is brought to the memory the first time, initialize reference bit to 0.
2. Advance the next victim pointer only if you need to find a victim page to replace, and when you bring a new page in.

1	2	3	4	4	3	2	1	5	6	2	1	2	3	7	8	3	2	1	5
1	1					1	1	1	1	1	1	1	1	1					
	2	2	2	2	2	2	2	2	2	2	2	2	2	2					
		3	3	3	3	3	3	3	3	3	3	3	3	3					
			4	4	4	4	4	4	6	6	6	6	6	7					

of page faults: H T T T
 # of page hits: H T T T T T
 # of page replacements: T T